

Security Assessment Report

Generated: 2026-05-05 13:04

Report ID: 225a0aa3

Security Assessment Report

Target: /workspace/uploads/ee1f2899c53c

Primary Component Analyzed: server.py

Assessment Sources: Bandit, SonarQube, Safety

1. Executive Summary

The automated assessment of the Python application identified a small number of code quality findings in server.py, with no issues reported by Bandit and no dependency analysis possible due to the absence of a requirements.txt file. SonarQube reported five issues, all classified as code smells rather than direct security vulnerabilities.

The most notable findings are three instances of excessive cognitive complexity in server.py, which indicate functions that are difficult to understand, test, and maintain. While these findings do not directly confirm exploitable security weaknesses, highly complex code can increase the probability of security defects being introduced during future changes and can reduce auditability of critical logic.

Overall system risk level: Low to Medium

Low from a direct exploitability perspective, because no confirmed security vulnerabilities were identified in the supplied scan results.

Medium from a maintainability and operational risk perspective, due to complex functions that may conceal logic flaws and hinder secure development practices.

2. Risk Overview

Findings by Severity

Critical: 3

High: 0

Medium: 0

Low: 2

Informational: 0

Risk Interpretation

Critical findings in this report are SonarQube cognitive complexity issues. These are not direct vulnerabilities, but they indicate sections of code that are harder to reason about and therefore more likely to harbor defects or security regressions.

Low findings relate to commentedout code. These are not security vulnerabilities but suggest code hygiene issues that can reduce maintainability and readability.

No confirmed exploitable security issues were identified in the provided scan output.

Dependency risk could not be assessed because Safety could not run without a requirements.txt file.

3. Detailed Findings

Critical Vulnerabilities

Finding 1 — Excessive Cognitive Complexity in Function at Line 120

Severity: Critical

CVSS Score (estimated): 6.5

OWASP Category:

A04: Insecure Design

A10: ServerSide Request Forgery / Other (indirectly related through maintainability and design risk, not a direct vulnerability)

Affected Component:

server.py — line 120

Description:

SonarQube flagged a function beginning at line 120 with a cognitive complexity score of 26, exceeding the allowed threshold of 15.

The function contains multiple nested control structures and branching paths, making it difficult to review, test, and safely modify.

While this is not a direct security flaw, complex functions are more prone to hidden logic errors and insecure edgecase handling.

Impact:

Attackers may benefit indirectly if future changes introduce authorization, validation, or input handling mistakes in this complex logic.

Business impact includes higher maintenance cost, slower incident response, and increased chance of regression in security sensitive code.

Proof of Concept (if possible):

Not directly exploitable based on scan output alone. The risk is maintainability driven rather than a confirmed attack path.

Recommended Remediation:

Refactor the function into smaller, single purpose helper functions.

Reduce nesting by using guard clauses and early returns where appropriate.

Isolate validation, business logic, and I/O operations.

Add unit tests for each decision branch to ensure behavior remains consistent after refactoring.

Finding 2 — Excessive Cognitive Complexity in Function at Line 151

Severity: Critical

CVSS Score (estimated): 6.5

OWASP Category:

A04: Insecure Design

Affected Component:

server.py — line 151

Description:

SonarQube reported a function at line 151 with cognitive complexity of 17, above the acceptable threshold.

The function includes several conditional branches and nested logic blocks, which increases the likelihood of implementation mistakes.

This complexity makes security review more difficult and can obscure unsafe behavior.

Impact:

Increased chance of logic defects in areas that may influence request handling, validation, or application control flow.

Potential business impact includes unstable application behavior and reduced confidence in code correctness during future feature changes.

Proof of Concept (if possible):

No direct exploit was demonstrated by the scan results.

Recommended Remediation:

Break the function into smaller routines with clear responsibilities.

Replace nested conditionals with structured decision handling where possible.

Introduce comprehensive tests around all execution paths.

Conduct a manual code review of the logic to confirm no hidden security-sensitive branches exist.

Finding 3 — Excessive Cognitive Complexity in Function at Line 198

Severity: Critical

CVSS Score (estimated): 6.5

OWASP Category:

A04: Insecure Design

Affected Component:

server.py — line 198

Description:

SonarQube flagged another function with cognitive complexity of 25.

The issue indicates a substantial amount of branching and nested logic, making the function difficult to understand and maintain.

Complex control flow can mask security flaws, especially in code that handles user input or orchestrates sensitive operations.

Impact:

Increased risk of introducing authentication, authorization, input validation, or statemanagement defects in future updates.

Operationally, the code becomes harder to troubleshoot, review, and securely extend.

Proof of Concept (if possible):

Not applicable; this is a structural code smell, not a directly exploitable issue from the provided evidence.

Recommended Remediation:

- Refactor into smaller functions and/or classes.
- Use explicit, readable control flow with minimal nesting.
- Separate business logic from infrastructure concerns.
- Add regression tests to preserve behavior during refactoring.

High Vulnerabilities

No highseverity vulnerabilities were identified in the provided scan results.

Medium Vulnerabilities

No mediumseverity vulnerabilities were identified in the provided scan results.

Low Vulnerabilities

Finding 4 — CommentedOut Code Present in server.py at Line 26

Severity: Low

CVSS Score (estimated): 0.0

OWASP Category:

A04: Insecure Design

A05: Security Misconfiguration

Affected Component:

server.py — line 26

Description:

SonarQube flagged commentedout code at line 26.

Commented code is generally considered poor code hygiene and can lead to confusion about intended functionality.

In some cases, stale commented code may leak implementation history or create uncertainty during review, but it is not a direct security issue on its own.

Impact:

Reduced readability and maintainability.

Potential exposure of obsolete logic that may confuse future maintainers.

Minor risk of accidentally reintroducing deprecated behavior.

Proof of Concept (if possible):

Not applicable.

Recommended Remediation:

Remove dead or commentedout code from the codebase.

If the code is needed for reference, use version control history rather than inline comments.

Keep source files clean to improve review quality.

Finding 5 — CommentedOut Code Present in server.py at Line 34

Severity: Low

CVSS Score (estimated): 0.0

OWASP Category:

A04: Insecure Design

A05: Security Misconfiguration

Affected Component:

server.py — line 34

Description:

A second instance of commentedout code was identified.

Similar to the first occurrence, this is primarily a maintainability issue.

It can obscure the intended logic of the application and make code reviews less effective.

Impact:

Reduced clarity in the affected module.

Increased chance of confusion during changes or security reviews.

Proof of Concept (if possible):

Not applicable.

Recommended Remediation:

Remove unnecessary commented code.

Document relevant design decisions in external documentation or commit history instead.

Informational Findings

No informational findings were reported in the scan results.

4. Recommendations (Global)

1. Refactor complex functions

Break large functions into smaller units with single responsibilities.

Reduce branching depth and eliminate unnecessary nesting.

2. Strengthen secure coding practices

Apply input validation and output encoding consistently.

Use explicit error handling and avoid ambiguous control flow.

3. Introduce automated testing

Add unit and integration tests covering all branches of complex functions.

Include negative test cases for malformed or unexpected inputs.

4. Improve dependency governance

Add a requirements.txt or equivalent dependency manifest.

Run dependency vulnerability scanning once dependencies are declared.

Keep thirdparty packages updated and pinned to approved versions.

5. Maintain code hygiene

Remove commentedout and dead code.

Apply linting and static analysis in CI/CD to prevent code smell accumulation.

6. Perform targeted manual review

Review the complex functions identified by SonarQube for hidden logic flaws.

Pay particular attention to any paths involving authentication, authorization, file handling, data parsing, or external requests.

5. Conclusion

The automated scan did not identify any confirmed exploitable security vulnerabilities in the provided results. However, the application contains several highcomplexity functions in `server.py`, which increase maintainability risk and may contribute to future security defects if not addressed. The presence of commentedout code further indicates the need for stronger code hygiene.

The most urgent remediation effort should focus on refactoring the critical complexity hotspots and establishing proper dependency management so that package vulnerabilities can be assessed. While the current direct security risk appears limited, improving code clarity and structure is important to reduce the likelihood of introducing serious issues over time.